
Trabajo Práctico Integrador

Redes y Comunicación

Implementación, red y despliegue del proyecto final

Materia:

Redes y Comunicación

Profesor:

Fernando Rossler

Proyecto Final del Grupo:

Sistema de Gestión Agrónoma "Don Elio"

Entrega Parcial:

Parte 1 y Parte 2 — 1er Cuatrimestre

Fecha de entrega:

23 de octubre de 2025

1. Introducción

"Don Elio" es un sistema de gestión agrónoma desarrollado como aplicación nativa Android, destinado a profesionales del sector agropecuario. Resuelve la necesidad concreta de un ingeniero agrónomo o productor rural de centralizar y digitalizar el seguimiento de sus actividades productivas, reemplazando el uso de planillas físicas o cuadernos.

El problema que resuelve: Los productores agropecuarios gestionan campañas agrícolas que involucran múltiples dimensiones —insumos, cosechas, tareas operativas y observaciones de campo— de manera desorganizada y sin trazabilidad. Don Elio unifica todo esto en una sola herramienta, permitiendo analizar el rendimiento económico de cada campaña y tomar decisiones informadas.

Tipo de organización destinataria: Empresas agropecuarias de pequeña y mediana escala, cooperativas rurales, estudios de ingeniería agronómica con múltiples clientes.

Alcance de la solución:

- Gestión completa de campañas agrícolas (creación, seguimiento, cierre).
- Registro de insumos con catálogo reutilizable y vinculación por campaña.
- Planificación de tareas con recordatorios y confirmación de ejecución.
- Registro de cosechas (almacenadas y vendidas/reservadas) con cálculo de rendimiento.
- Observaciones de campo con soporte de fotografías.
- Reportes comparativos entre campañas con exportación a CSV y PDF.
- Autenticación por usuario con contraseña hasheada (SHA-256).
- Backup y restauración de la base de datos local.

Parte 1: Configuración de la Red e Infraestructura de Implementación

En esta sección se diseña la infraestructura de red necesaria para alojar y poner en funcionamiento el sistema Don Elio en distintos escenarios reales de implementación. La propuesta contempla cómo se implementaría la solución considerando recursos disponibles, cantidad de usuarios, seguridad, mantenimiento y posibilidad de crecimiento.

1.1. Diseño y Topología de la Red

Elección de topología: Estrella con Switch Central

Para implementar Don Elio en un establecimiento agropecuario se propone una **topología en estrella**, donde todos los dispositivos se conectan a un switch central que a su vez se vincula al router de acceso a Internet.

Justificación técnica para el contexto de Don Elio:

- **Aislamiento de fallas:** Si una tablet del capataz o la PC de la oficina falla, el resto de la red —incluido el servidor que aloja los backups y reportes— no se ve afectado. Esto es crítico durante la temporada de cosecha, donde la continuidad operativa es prioritaria.
- **Escalabilidad progresiva:** En una primera etapa el establecimiento puede tener 3 dispositivos. En el futuro, al incorporar más operarios o sectores (ganadería, logística), agregar nodos es simplemente conectar un cable al switch.
- **Administración centralizada:** El switch permite monitorear el tráfico, aplicar VLAN (en una versión administrable) y segmentar la red entre usuarios de campo y administración.
- **Bajo costo:** El hardware necesario (switch no administrado, router doméstico o SOHO) es accesible para una empresa agropecuaria mediana.

Limitación y riesgo: El switch central es un único punto de falla. En el escenario de empresa grande se mitigará con redundancia de enlaces y switches en cascada.

Escenario A — Cliente Pequeño (Establecimiento rural, 3–5 usuarios)

La app Don Elio corre directamente en dispositivos Android. Para el escenario pequeño, el "servidor" es una PC o NUC de bajo consumo que actúa como servidor de archivos para los backups de la base de datos SQLite y como servidor web liviano para un futuro panel de reportes.

```
Internet (ISP Rural / 4G)
|
[Router/Modem] 192.168.1.1
|
[Switch 8 puertos]
|-- [PC Servidor de Backup] 192.168.1.10 (Ubuntu / Win 10 Pro)
|-- [Tablet Capataz] 192.168.1.20 (App Don Elio - Android)
|-- [Smartphone Agronomo] 192.168.1.21 (App Don Elio - Android)
+-- [PC Administracion] 192.168.1.22 (Acceso a reportes)
```

En este escenario, la base de datos Room (SQLite) reside en el dispositivo Android de cada usuario. Los backups se exportan manualmente al servidor de archivos mediante la funcionalidad ya implementada (*CrearBackupUseCase*). La sincronización entre dispositivos no es en tiempo real: cada operario trabaja con su copia local y se reconcilian mediante backup/restauración.

Escenario B — Empresa Grande (Cooperativa o estudio agronómico, 20+ usuarios)

En este escenario, Don Elio evolucionaría incorporando una **API REST centralizada** que reemplaza la base de datos local por una base de datos compartida en servidor. La topología cambia a una arquitectura cliente-servidor distribuida con segmentación por VLANs:

```
Internet
|
[Firewall / Router empresarial]
|
[Switch Core - Capa 3]
|-- VLAN 10 Administracion [Servidores App + BD] 192.168.10.x/24
|-- VLAN 20 Operaciones [Tablets y smartphones] 192.168.20.x/24
+-- VLAN 30 Acceso remoto [VPN endpoint] 192.168.30.x/24
```

Cambios clave para este escenario:

- Servidor de aplicación (API REST) separado del servidor de base de datos.
- Balanceador de carga para soportar múltiples instancias de la API.

- VPN para acceso de agrónomos externos sin exponer el servidor a Internet.
- Monitoreo con herramientas como Zabbix o Grafana.

Plan de Direccionamiento IP

Se define un rango de red privada **192.168.1.0/24**. Los dispositivos críticos (servidor) tienen IP estática; los clientes pueden usar DHCP con reserva por dirección MAC.

Dispositivo / Servicio	Dirección IP	Máscara	Gateway	Función en Don Elio
Router / Gateway	192.168.1.1	255.255.255.0	N/A	Salida a Internet
PC Servidor de Backup	192.168.1.10	255.255.255.0	192.168.1.1	Backups .db y panel de reportes
Tablet Capataz (Android)	192.168.1.20	255.255.255.0	192.168.1.1	App Don Elio - tareas y cosechas
Smartphone Agrónomo (Android)	192.168.1.21	255.255.255.0	192.168.1.1	App Don Elio - visualiza reportes
PC Administración	192.168.1.22	255.255.255.0	192.168.1.1	Descarga exportaciones CSV/PDF

¿Por qué IP estática para el servidor? Los dispositivos Android que exportan backups apuntan siempre a la misma dirección. Si la IP cambiara (DHCP), los scripts de sincronización fallarían. La IP estática garantiza que 192.168.1.10 sea siempre el servidor de backup, sin necesidad de reconfigurar los dispositivos Android.

1.2. Configuración Teórica del Servidor o Entorno de Ejecución

Sistema Operativo: Ubuntu Server 24.04 LTS

Justificación: gratuito y sin costos de licencia; soporte LTS de 5 años (hasta 2029); compatibilidad nativa con Node.js, SQLite, MySQL y Nginx; estable en hardware de bajo consumo (Raspberry Pi 4 o NUC Intel). Para el escenario de cliente pequeño con Windows, también es válido Windows 10/11 Pro con XAMPP o Laragon.

Servidor Web: Nginx

En la versión actual de Don Elio (app móvil pura), Nginx cumple la función de servir el panel de reportes exportados (archivos HTML/PDF/CSV) y actuar como proxy inverso en caso de agregar una API REST en el futuro.

Instalación conceptual en Ubuntu Server:

```
sudo apt update && sudo apt upgrade -y
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx
```

Base de Datos: SQLite (local en Android) / MySQL 8.0 (servidor empresa grande)

Don Elio utiliza SQLite a través de Room (ORM de Android). No requiere servidor separado: la base de datos es un único archivo *.db* que reside en el dispositivo. Room genera todo el acceso SQL automáticamente, soporta las 8 tablas relacionales del sistema con Foreign Keys y migraciones versionadas (actualmente en versión 5). El archivo *.db* se exporta como backup con la funcionalidad ya implementada.

Para el escenario de empresa grande con múltiples usuarios concurrentes, SQLite se reemplaza por **MySQL 8.0**, que soporta escrituras concurrentes y ofrece mayor robustez y herramientas de administración.

Variables de Entorno y Configuración

Los siguientes datos no deben estar en el código fuente:

```
# Configuración de la API REST (escenario empresa grande)
DB_HOST=192.168.1.11
DB_PORT=3306
DB_NAME=donelio_produccion
DB_USER=donelio_app
DB_PASSWORD=<contrasena_segura>
JWT_SECRET=<clave_aleatoria_256_bits>
BACKUP_DIR=/var/backups/donelio
```

Diseño de Datos — Entidades Principales de Don Elio

A continuación se presentan las entidades centrales del sistema, extraídas directamente del código fuente de la aplicación (capa *data/local/entity/*).

*Tabla **campanias** — Entidad central del sistema:*

```
CREATE TABLE campanias (
  id_campania INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT NOT NULL,
  fecha INTEGER NOT NULL, -- Timestamp en milisegundos
```

```
cultivo TEXT NOT NULL,  
esta_activa INTEGER NOT NULL DEFAULT 1 -- 1=activa, 0=cerrada  
);
```

Tabla insumos — Catálogo de insumos agrícolas:

```
CREATE TABLE insumos (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
nombre TEXT NOT NULL,  
categoria TEXT NOT NULL, -- 'Semilla', 'Fertilizante', 'Herbicida'  
icono TEXT, -- nombre del icono Material para la UI  
activo INTEGER NOT NULL DEFAULT 1  
);
```

Tabla campania_insumos — Vinculación insumo ↔ campaña:

```
CREATE TABLE campania_insumos (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
id_campania INTEGER NOT NULL,  
id_insumo INTEGER NOT NULL,  
cantidad REAL NOT NULL,  
precio REAL NOT NULL,  
FOREIGN KEY (id_campania) REFERENCES campanias(id_campania) ON DELETE CASCADE,  
FOREIGN KEY (id_insumo) REFERENCES insumos(id) ON DELETE RESTRICT  
);
```

Nota: ON DELETE RESTRICT en insumos previene eliminar un insumo en uso en una campaña activa.

Tabla tareas — Actividades planificadas:

```
CREATE TABLE tareas (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
nombre TEXT NOT NULL,  
fecha INTEGER NOT NULL, -- Timestamp normalizado a 00:00:00  
hora TEXT NOT NULL,  
notificar INTEGER NOT NULL DEFAULT 0,  
confirmar INTEGER NOT NULL DEFAULT 0,  
id_campania INTEGER NOT NULL,  
FOREIGN KEY (id_campania) REFERENCES campanias(id_campania) ON DELETE CASCADE  
);
```

Tabla cosechas — Registro de producción:

```
CREATE TABLE cosechas (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
cantidad REAL NOT NULL,  
fecha INTEGER NOT NULL,  
almacen TEXT NOT NULL,  
id_campania INTEGER NOT NULL,  
FOREIGN KEY (id_campania) REFERENCES campanias(id_campania) ON DELETE CASCADE  
);
```

Tabla usuarios — Autenticación:

```
CREATE TABLE usuarios (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre TEXT NOT NULL,  
  nombre_usuario TEXT NOT NULL UNIQUE,  
  contrasena TEXT NOT NULL, -- Hash SHA-256 de la contrasena  
  ultimo_acceso INTEGER -- Timestamp del ultimo login  
);
```

1.3. Simulación de Pruebas de Conectividad y Funcionamiento

Aunque no se ejecutan los comandos en un entorno físico, se describe qué pruebas se realizarían y cuál sería el resultado esperado.

Prueba 1 — ping: verificar conectividad entre cliente y servidor

Propósito: Confirmar que la tablet del capataz puede alcanzar el servidor de backup.

```
ping 192.168.1.10
```

Salida esperada:

```
PING 192.168.1.10 56(84) bytes of data.  
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=1.23 ms  
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=0.98 ms  
--- 192.168.1.10 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss
```

Interpretación: 0% de pérdida y latencia < 5 ms confirman que la ruta LAN está correcta y los backups podrán transferirse sin corrupción.

Prueba 2 — traceroute: diagnóstico de ruta de paquetes

Propósito: Verificar que el tráfico entre la tablet y el servidor pasa solo por el switch (un salto), sin atravesar el router innecesariamente.

```
traceroute 192.168.1.10
```

Salida esperada:

```
traceroute to 192.168.1.10, 30 hops max  
1 192.168.1.10 (192.168.1.10) 1.230 ms 0.980 ms 1.050 ms
```

Un solo salto indica que ambos dispositivos están en la misma subred /24, comunicándose directamente mediante el switch sin necesidad del router.

Prueba 3 — Servicio web (panel de reportes)

Desde la PC de Administración, verificar que Nginx responde:

```
curl -I http://192.168.1.10  
# Esperado: HTTP/1.1 200 OK
```

Prueba 4 — Base de datos: integridad del backup

Exportar el archivo *.db* desde la app (Configuración → Exportar backup) y verificar su integridad en el servidor:

```
sqlite3 donelio_backup_20251023.db "SELECT COUNT(*) FROM campanias;"  
# Resultado esperado: numero de campanias registradas (ej: 4)
```

Prueba 5 — Multiusuario

El agrónomo (*donelio_admin*) y el capataz (*capataz_campo*) acceden a la app simultáneamente desde distintos dispositivos. Cada dispositivo tiene su propia base de datos Room local, por lo que no hay conflictos de escritura concurrente. El *LoginUseCase* valida que las credenciales de cada usuario son independientes mediante SHA-256.

Parte 2: Desarrollo de la Aplicación — Sistema Don Elio

En esta sección se describe la aplicación, su arquitectura y su lógica, tomando como referencia directa el proyecto final desarrollado. Se incluyen pseudocódigo, mockups, diagramas, descripciones de módulos, flujos de usuario y justificaciones técnicas.

Stack Tecnológico

Don Elio fue desarrollado con el siguiente stack:

Capa	Tecnología	Rol
Lenguaje	Kotlin 2.x	Lenguaje principal de la app
UI	Jetpack Compose + Material 3	Interfaz declarativa y reactiva
Base de datos	Room (SQLite)	Persistencia local offline
Inyección de dep.	Dagger-Hilt 2.x	DI automática entre capas
Concurrencia	Coroutines + Flow	Asincronía y streams reactivos
Navegación	Navigation Compose	Ruteo entre pantallas
Gráficos	YCharts	PieChart en pantalla Reportes
Imágenes	Coil Compose	Carga de fotos de observaciones
Tareas diferidas	WorkManager	Recordatorios de tareas
Preferencias	DataStore	Almacenamiento de sesión
Testing	JUnit 4 + MockK + Turbine	Pruebas unitarias e instrumentadas
Cobertura	Kover	Reporte de cobertura de código

Justificación del stack:

- **Kotlin nativo:** El contexto de uso es campo abierto con conectividad limitada. Una app Android nativa funciona sin Internet, al contrario de una solución web.
- **Room sobre SQLite:** El entorno Android ya incluye SQLite; Room añade seguridad de tipos, consultas en Flow reactivo y migraciones controladas.
- **Hilt:** Reduce el boilerplate de inyección de dependencias y se integra nativamente con ViewModel y Navigation Compose.
- **Coroutines + Flow:** Permiten que la UI reaccione automáticamente a cambios en la BD sin polling manual. Por ejemplo, al agregar una cosecha, el dashboard se actualiza

solo.

Arquitectura del Sistema

Don Elio implementa **Clean Architecture** con separación estricta en tres capas más el módulo de inyección de dependencias:

```
■ PRESENTATION (UI / ViewModel) ■  
■ Jetpack Compose + StateFlow + Navigation Compose ■  
  
■ invoca  
  
■ DOMAIN (Use Cases / Modelos puros) ■  
■ Logica de negocio – sin Android, sin Room ■  
■ 35 casos de uso – data class puros de Kotlin ■  
  
■ implementa interfaces  
  
■ DATA (Room / DAOs / Mappers) ■  
■ Entidades Room -> Mappers -> Modelos de dominio ■  
■ 8 tablas SQLite – 8 DAOs – Migraciones v1 -> v5 ■
```

Patrón de flujo de datos (unidireccional):

```

UI Screen --> ViewModel --> Use Case --> Repository Interface
|
Repository Implementation
|
DAO (Room)
|
SQLite (.db file)

```

El flujo de retorno usa *StateFlow* y *Flow<Resource<T>>* que propagan automáticamente los cambios desde SQLite hasta la UI sin polling manual.

Módulos Principales del Sistema

Módulo	ViewModels principales	Use Cases principales
Autenticación	LoginViewModel	LoginUseCase, RegistroUseCase
Campañas	GestionCampaniasVM, CampaniaDetailVM, CampaniaFormVM	CrearCampaniaUC, EditarCampaniaUC, EliminarCampaniaUC, FinalizarCampaniaUC

Módulo	ViewModels principales	Use Cases principales
Insumos	InsumoCatalogoVM, InsumoVinculacionVM, FormularioInsumoVM	CrearInsumoCatalogoUC, AsignarInsumoACampaniaUC, DesvincularInsumoUC
Tareas	TareaViewModel, NuevaTareaViewModel	CrearTareaUC, ConfirmarTareaUC, EditarTareaUC, EliminarTareaUC
Cosechas	CosechaViewModel, FormularioCosechaVM	RegistrarCosechaUC, RegistrarCosechaConVentaUC
Observaciones	ObservacionVM, FormularioObservacionVM	GuardarObservacionUC, ObtenerObservacionesPorCampaniaUC
Reportes	ReportesViewModel	ObtenerCampaniasUC, ObtenerCosechasPorCampaniaUC, ObtenerInsumosVinculadosUC
Config / Backup	ConfiguracionDBVM, BackupViewModel	CrearBackupUC, RestaurarBackupUC

Usuarios y Permisos

Tipo de usuario	Acceso	Capacidades
Usuario registrado	Login con contraseña	Acceso completo: crear campañas, registrar cosechas, gestionar insumos, exportar reportes, realizar backups
Invitado (debug)	Sin contraseña	Solo disponible en builds de debug. Acceso limitado a lectura

Las contraseñas se almacenan como hash SHA-256 (función `hashContrasena()` en `LoginUseCase.kt`). No existe recuperación de contraseña en la versión actual.

Diseño de la Interfaz — Vistas Principales

A continuación se presentan los wireframes de las pantallas principales del sistema.

Vista 1 — Dashboard / Pantalla Principal:

```
+-----+
| Don Elio Buenas tardes, Martin |
| ----- |
| Campana Activa |
| +-----+ |
| | Soja 2025/2026 | | |
| | Inicio: 01/10/2025 | Activa | |
| | Tareas pendientes: 3 | |
| +-----+ |
| |
```

```

| Mis Campanias [+ Nueva] |
| +-----+ |
| | Maiz 2024/2025 FINALIZADA | |
| | Soja 2025/2026 ACTIVA | |
| +-----+ |
| [Inicio] [Tareas] [Reportes] |
| +-----+

```

Vista 2 — Detalle de Campaña (5 tabs):

```

+-----+
| <- Soja 2025/2026 Activa |
| Inicio: 01/10/2025 |
| [Info][Tareas][Insumos][Cosechas][Obs] |
| ----- |
| INSUMOS |
| +-----+ |
| | Soja RR DM 5.9i 500 kg | |
| | $ 450.000 | |
| | Glifosato 48% 200 L | |
| | $ 180.000 | |
| | Total insumos: $ 630.000 | |
| +-----+ |
| [+ Agregar] |
| +-----+

```

Vista 3 — Pantalla de Reportes:

```

+-----+
| Reportes de Rendimiento [Exportar] |
| Campana: [ Soja 2025/2026 (v) ] |
| ----- |
| Costo total de insumos: $ 630.000 |
| Total cosechado: 180 tn |
| |
| Destino de Cosechas (PieChart): |
| Almacenado 60% |
| Vendido 30% |
| Reservado 10% |
| ----- |
| Comparador: Campana A vs Campana B |
| +-----+

```

Lógica de la Aplicación — Pseudocódigo de Funcionalidades Clave

Funcionalidad 1 — Crear Campaña

Recibe: nombre: String, cultivo: String, fechaInicio: Long

Devuelve: Flow<Resource<Unit>> (Loading → Success | Error)

```
FUNCION crearCampania(nombre, cultivo, fechaInicio)

SI nombre esta vacio ENTONCES
EMITIR Resource.Error("El nombre no puede estar vacio")
TERMINAR
FIN SI

campania = Campania(
id = 0, -- Room asigna el ID real al insertar
nombre = nombre.trim(),
cultivo = cultivo.trim(),
fechaInicio = fechaInicio,
estaActiva = true
)

EMITIR Resource.Loading
EN HILO IO:
campaniaRepository.insertCampania(campania)
EMITIR Resource.Success

FIN FUNCION
```

Funcionalidad 2 — Registrar Cosecha con Venta

Recibe: cantidad: Double, fecha: Long, almacen: String, idCampania: Int, esVenta: Boolean, precio: Double?

Devuelve: Unit (lanza excepción en caso de error)

```
FUNCION registrarCosechaConVenta(cantidad, fecha, almacen, idCampania, esVenta,
precio)

SI cantidad <= 0 ENTONCES
LANZAR Error("La cantidad debe ser mayor a cero")
FIN SI

idCosecha = cosechaRepository.insertCosecha(
Cosecha(cantidad, fecha, almacen, idCampania)
)

SI esVenta ENTONCES
cosechaNoAlmacenadaRepository.insert(
CosechaNoAlmacenada(
precio = precio,
idCosecha = idCosecha -- FK hacia la cosecha base
)
)
)
```

```
FIN SI  
  
FIN FUNCION
```

Funcionalidad 3 — Autenticación de Usuario

Recibe: nombreUsuario: String, contraseña: String

Devuelve: Usuario? (null si las credenciales son incorrectas)

```
FUNCION login(nombreUsuario, contraseña)  
  
SI nombreUsuario vacio 0 contraseña vacia ENTONCES  
  LANZAR Error("Campos obligatorios")  
FIN SI  
  
hashIngresado = SHA256(contraseña.toByteArray(UTF-8))  
usuarioEntity = usuarioDao.getUsuarioByNombre(nombreUsuario.trim())  
  
SI usuarioEntity es null 0 contraseña != hashIngresado ENTONCES  
  DEVOLVER null -- Credenciales incorrectas  
SINO  
  DEVOLVER usuarioEntity.toDomain() -- Usuario autenticado  
FIN SI  
  
FIN FUNCION
```

Funcionalidad 4 — Confirmar Tarea Realizada

```
FUNCION confirmarTarea(tarea)  
  
SI tarea.confirmar == true ENTONCES  
  EMITIR Resource.Error("La tarea ya fue confirmada")  
  TERMINAR  
FIN SI  
  
tareaActualizada = tarea.copy(confirmar = true)  
EN HILO IO:  
  tareaRepository.updateTarea(tareaActualizada)  
  EMITIR Resource.Success  
  
FIN FUNCION
```

Uso de Librerías y Frameworks

Room — Base de Datos Local (ORM)

Capa de abstracción sobre SQLite que genera el código SQL en tiempo de compilación, detectando errores de consulta antes de ejecutar la app.

```
// DatabaseModule.kt (Hilt)
@Provides @Singleton
fun provideDatabase(@ApplicationContext context: Context): DonElioDatabase {
    return Room.databaseBuilder(context, DonElioDatabase::class.java, "donelio.db")
        .addMigrations(MIGRATION_4_5)
        .build()
}
```

Hilt — Inyección de Dependencias

Automatiza la creación y provisión de objetos complejos (DAOs, repositorios, casos de uso). La anotación `@HiltViewModel` en los ViewModels y `@Inject constructor` en los Use Cases permiten que Hilt resuelva el árbol de dependencias completo automáticamente.

Coroutines + Flow — Asincronía Reactiva

Permite ejecutar operaciones de base de datos en segundo plano (*Dispatchers.IO*) y emitir los resultados como un stream reactivo (*Flow*) que la UI escucha sin bloquear el hilo principal. Patrón usado: *Flow<Resource<T>>* con estados Loading → Success | Error.

WorkManager — Recordatorios de Tareas

Programa notificaciones en segundo plano garantizando su ejecución incluso si la app está cerrada. El *CrearTareaUseCase* invoca *taskReminderScheduler.schedule(tarea)* cuando el usuario activa la opción de notificación.

Manejo de Errores

Escenario de error	Respuesta del sistema
Campo obligatorio vacío	Se emite Resource.Error con mensaje descriptivo; la UI muestra el error bajo el campo afectado
Cantidad de cosecha ≤ 0	IllegalArgumentException capturada en el Use Case; la UI muestra Snackbar con el mensaje
Error de base de datos (ej. FK violation)	Resource.Error con e.localizedMessage; la UI muestra Snackbar genérico
Permiso de cámara denegado	EstadoPermisoCamara detecta la denegación y muestra DialogoRazonPermisoCamara o botón "Abrir Ajustes"

Escenario de error	Respuesta del sistema
Insumo en uso al eliminar	FK ON DELETE RESTRICT lanza excepción SQLite; el ViewModel la captura y muestra mensaje al usuario

Aspectos Transversales: Implementación, Escalabilidad y Mantenimiento

Instalación en Cliente Pequeño (Establecimiento rural, 1–5 usuarios)

Requisitos de hardware:

- Dispositivo Android con API 26+ (Android 8.0 Oreo o superior).
- Mínimo 2 GB de RAM; recomendado 4 GB.
- 100 MB de almacenamiento libre para la app y la base de datos.
- Cámara trasera (opcional, para fotos de observaciones).

Pasos de instalación:

- Obtener el APK de Don Elio o instalarlo desde la Play Store.
- Habilitar "Fuentes desconocidas" en el dispositivo Android si se instala por APK directo.
- Instalar el APK: `adb install app-release.apk` o tocar el archivo desde el explorador.
- Al primer inicio, Room crea la base de datos SQLite automáticamente.
- Crear el primer usuario desde la pantalla de Registro.

No se requiere servidor, conexión a Internet ni configuración de red para el funcionamiento básico de la app.

Implementación en Empresa Grande (Cooperativa, 20+ usuarios)

La arquitectura evoluciona de "app local" a "sistema cliente-servidor":

- **API REST centralizada** (Node.js + Express o Spring Boot): la app Android pasa a ser un cliente HTTP que consulta endpoints (`/api/campanias`, `/api/cosechas`).
- **Base de datos centralizada** (MySQL 8.0): las 8 tablas de Don Elio migran de SQLite a MySQL con el mismo esquema relacional.
- **Autenticación con tokens JWT**: el `LoginUseCase` actual (SHA-256 local) se reemplaza por un endpoint `/api/auth/login` que devuelve un token de 24 horas.

- **Permisos por rol:** tabla roles (Administrador, Agrónomo, Operario) con acciones permitidas por módulo.
- **Auditoría:** columnas creado_por y modificado_en en todas las tablas.

Escalabilidad

Componente actual	Limitación	Solución escalable
SQLite local por dispositivo	No permite compartir datos entre dispositivos	Migrar a MySQL/PostgreSQL en servidor centralizado
App Android monolítica	No accesible desde PC/navegador	Desarrollar API REST + cliente web (React/Vue)
Login SHA-256 sin tokens	Sin expiración de sesión	JWT con refresh tokens
Backups manuales	Sin automatización	Cron job en servidor con mysqldump automático
Un proceso de app	Sin distribución de carga	Balanceador (Nginx) + múltiples instancias de la API

Backups y Recuperación

Implementación actual (cliente pequeño):

- *CrearBackupUseCase* copia el archivo .db de Room al almacenamiento externo del dispositivo usando el Storage Access Framework (SAF).
- *RestaurarBackupUseCase* permite seleccionar un .db previo y reemplazar la base de datos actual.

Para empresa grande — backup automatizado:

```
# Cron job: backup diario a las 2:00 AM
0 2 * * * mysqldump -u donelio_app -p donelio_produccion | \
gzip > /var/backups/donelio/backup_$(date +%Y%m%d).sql.gz

# Retención: mantener solo los ultimos 30 dias
find /var/backups/donelio/ -mtime +30 -delete
```

Mantenimiento

- **Actualizaciones de la app:** Incrementar versionCode en build.gradle.kts, crear una migración Room si se modifican tablas (patrón ya implementado, actualmente en versión 5).

- **Registro de errores:** Integrar Firebase Crashlytics para captura automática de excepciones en producción.
- **Ciclo de vida de la BD:** Cada cambio de esquema requiere una migración explícita. Nunca usar `fallbackToDestructiveMigration()` en producción.

Seguridad

- **Autenticación:** Login con hash SHA-256 de la contraseña. Nunca se almacena en texto plano. Mejora futura: agregar salting para mayor robustez.
- **Protección de datos:** La base de datos SQLite está en el directorio privado de la app (`/data/data/com.itec.donelio/databases/`), inaccesible para otras apps sin root.
- **Permisos mínimos:** El `AndroidManifest.xml` declara solo `CAMERA` y `POST_NOTIFICATIONS`. No se solicitan permisos de red, ubicación ni contactos.
- **Prevención de inyecciones:** Room usa internamente Prepared Statements, eliminando el riesgo de SQL Injection en todas las consultas generadas.
- **Comunicación segura:** Para el acceso remoto futuro, usar HTTPS con certificado SSL/TLS y Port Forwarding restringido al puerto 443.

Conclusión

El sistema Don Elio demuestra cómo un proyecto final universitario puede pensarse con la misma rigurosidad técnica que un sistema productivo real. A lo largo de este trabajo se analizaron:

- La **infraestructura de red** necesaria en dos escenarios reales: establecimiento rural pequeño (red en estrella, servidor de backup local, SQLite por dispositivo) y empresa agropecuaria grande (VLANs segmentadas, API REST centralizada, MySQL, autenticación JWT).
- La **arquitectura del sistema** (Clean Architecture con Kotlin, Room, Hilt y Compose), que aplica los mismos principios de separación de responsabilidades estudiados en materias de Ingeniería de Software.
- Las **lógicas de negocio clave** documentadas como pseudocódigo comprensible.
- Los **aspectos transversales** de todo sistema productivo: instalación, escalabilidad, backup, mantenimiento y seguridad.

El resultado es un plano técnico completo que podría entregarse a un cliente real o a un equipo de desarrollo, demostrando no solo cómo se desarrolla la aplicación, sino cómo se instala, se conecta, se protege, se mantiene y está preparada para crecer.

Bibliografía

- Android Developers. (2024). *Room persistence library*. Google.
<https://developer.android.com/training/data-storage/room>
- Android Developers. (2024). *Jetpack Compose documentation*. Google.
<https://developer.android.com/jetpack/compose>
- Android Developers. (2024). *Hilt dependency injection*. Google.
<https://developer.android.com/training/dependency-injection/hilt-android>
- Android Developers. (2024). *WorkManager*. Google.
<https://developer.android.com/topic/libraries/architecture/workmanager>
- Nginx, Inc. (2024). *Nginx documentation*. <https://nginx.org/en/docs/>
- Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*.
<https://dev.mysql.com/doc/refman/8.0/en/>
- OWASP Foundation. (2024). *SQL Injection Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Foundation. (2024). *Password Storage Cheat Sheet*.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Prentice Hall.
- Ubuntu Community. (2024). *UFW - Uncomplicated Firewall*.
<https://help.ubuntu.com/community/UFW>

Anexos

Anexo A — Diagrama de Red

Generar con draw.io o diagrams.net siguiendo los esquemas de la Sección 1.1. El diagrama del escenario pequeño debe incluir: nube de Internet, router, switch central, PC servidor (192.168.1.10), dos dispositivos Android (192.168.1.20 y .21) y PC de administración (192.168.1.22).

Anexo B — Fragmento de Código Fuente Real: CrearCampaniaUseCase.kt

```
class CrearCampaniaUseCase @Inject constructor(
    private val campaniaRepository: CampaniaRepository
) {
    operator fun invoke(
        nombre: String,
        cultivo: String,
        fechaInicio: Long
    ): Flow<Resource<Unit>> = flow {
        emit(Resource.Loading)
        try {
            if (nombre.isBlank()) {
                throw IllegalArgumentException("El nombre no puede estar vacio")
            }
            val campania = Campania(
                id = 0, nombre = nombre.trim(),
                fechaInicio = fechaInicio, estaActiva = true,
                cultivo = cultivo.trim()
            )
            campaniaRepository.insertCampania(campania)
            emit(Resource.Success(Unit))
        } catch (e: Exception) {
            emit(Resource.Error(e.localizedMessage ?: "Error desconocido", e))
        }
    }.flowOn(Dispatchers.IO)
}
```

Anexo C — Fragmento de Código Fuente Real: LoginUseCase.kt

```
class LoginUseCase @Inject constructor(private val usuarioDao: UsuarioDao) {
    suspend operator fun invoke(nombreUsuario: String, contrasena: String): Usuario? {
        if (nombreUsuario.isBlank()) throw IllegalArgumentException("Usuario vacio")
        if (contrasena.isBlank()) throw IllegalArgumentException("Contrasena vacia")

        val hash = hashContrasena(contrasena)
        val entity = usuarioDao.getUsuarioByNombre(nombreUsuario.trim())
    }
}
```

```
return if (entity != null && entity.contrasena == hash)
entity.toDomain() else null
}
}

fun hashContrasena(contrasena: String): String {
val digest = MessageDigest.getInstance("SHA-256")
val bytes = digest.digest(contrasena.toByteArray(Charsets.UTF_8))
return bytes.joinToString("") { "%02x".format(it) }
}
```